UNIVERSIDAD NACIONAL

Sede Regional Chorotega, Campus Nicoya

Carrera: Ingeniería en Sistemas de la Información

Seguridad Informática

Laboratorio 5

UNA Chat: prevención de inyección de scripts, URLs de imágenes y videos, pruebas unitarias, de integración y end-to-end

Repositorio de GitHub

https://github.com/Veldor09/lab5SI

Estudiante: Diego Araya

Persona académica: Ing. Alex Villegas Carranza, M.Sc.

Fecha de clase: viernes 16 de octubre de 2026

Fecha de entrega: viernes 23 de octubre de 2026

Costa Rica, Guanacaste, 2026

1. Introducción y estructura del repositorio

En este laboratorio se ejecutó el proyecto **UNA Chat** (Node.js, Express y Socket.IO), se comprobó que era vulnerable a la inyección de scripts (XSS) y se corrigió la validación de mensajes en la librería **una-lib**. Además, se agregó la funcionalidad para mostrar imágenes y videos a partir de URLs. El trabajo se verificó con tres tipos de pruebas automatizadas: unitarias, de integración y end-to-end.

Todo el código está en el repositorio https://github.com/Veldor09/lab5SI, organizado así:

Ruta	Contenido
`antes/`	Código original y vulnerable de UNA Chat, conservado para reproducir la evidencia 1.
`despues/libs/unalib.js`	Librería una-lib con la nueva validación `validarMensaje`.
`despues/server.js`	Backend Express + Socket.IO; valida cada mensaje con una-lib antes de reenviarlo.
`despues/index.html`	Frontend del chat.
`despues/tests/unit/`	Pruebas unitarias (Jest).
`despues/tests/integration/`	Pruebas de integración backend + una-lib (Jest, Supertest y socket.io-client).
`despues/tests/e2e/` y `playwright.config.js`	Pruebas end-to-end con Playwright en Chromium.
`evidencias/`	Capturas de pantalla en JPG.
`repositorio.txt`	Enlace al repositorio.

Comandos para ejecutar el proyecto y las pruebas

```
cd despues
npm install
npx playwright install chromium
npm start                  # chat en http://localhost:3000
npm test                   # pruebas unitarias
npm run test:integration   # npx jest tests/integration --runInBand --env=node
npm run test:e2e           # npx playwright test --project=chromium --headed
npx playwright show-report # reporte HTML de Playwright
```

> En Windows PowerShell, si la política de ejecución bloquea `npm`, se usa `npm.cmd` y `npx.cmd`, como se ve en las capturas.

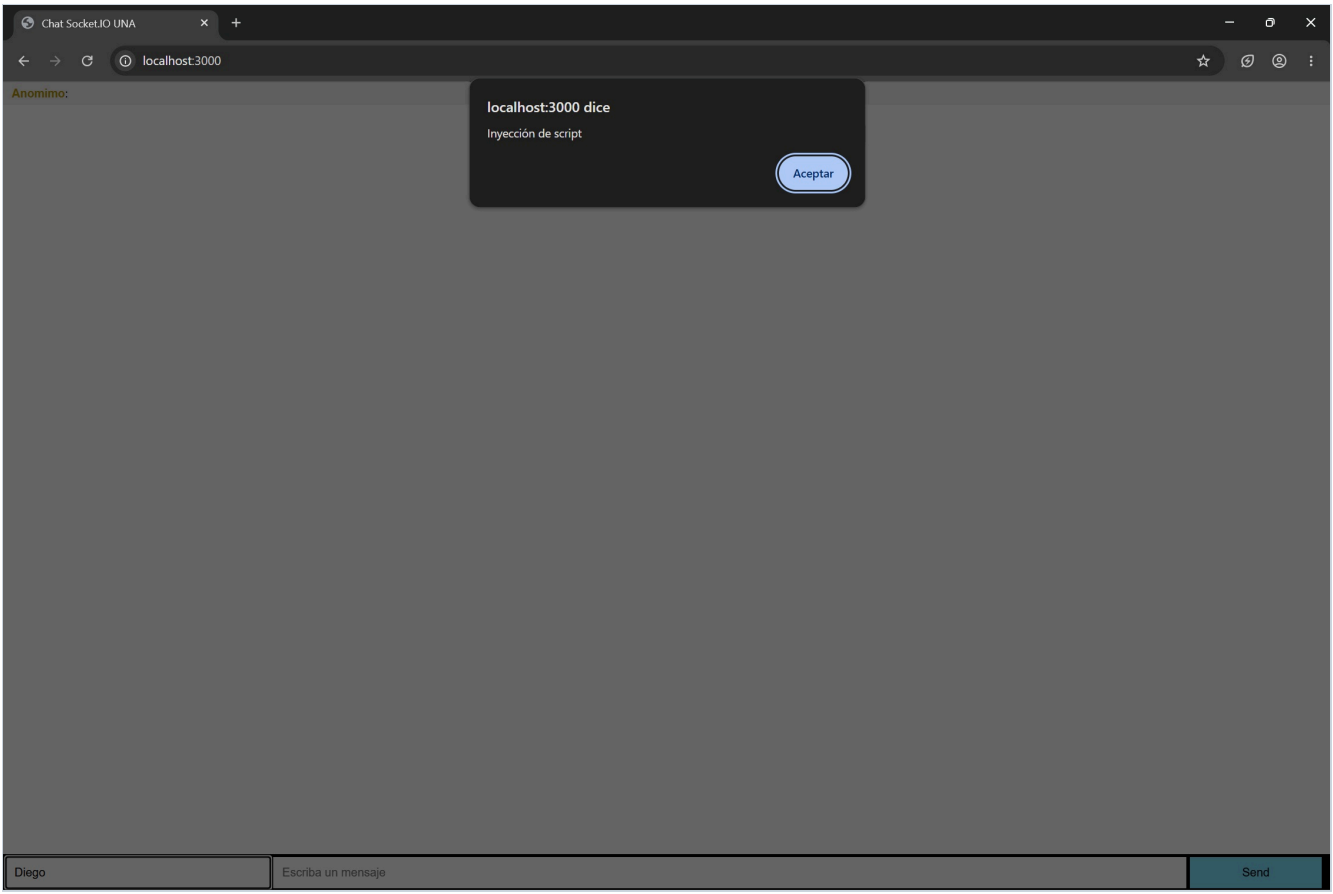
2. Ejecución del proyecto y prueba de inyección de scripts (pasos 1 a 3)

Se instalaron las dependencias con `npm install` y se inició la aplicación original con `npm start`. El chat quedó disponible en `http://localhost:3000`. En el campo de mensaje se envió el script del enunciado:

```
<script>alert('Inyección de script');</script>
```

El navegador mostró el cuadro de alerta, lo que confirma que **la aplicación original es vulnerable a la inyección de scripts**. El análisis del código mostró tres causas:

- El frontend inserta el mensaje con `$('<li>').html(...)` de jQuery, que interpreta el texto como HTML y ejecuta las etiquetas `<script>`.
- `validateMessage` no escapaba nada: devolvía el texto recibido tal cual, también el nombre de usuario y el color.
- Las funciones `is_valid_url_image` e `is_valid_yt_video` usaban una variable inexistente (`phone`), por lo que lanzaban un error y nunca reconocían imágenes ni videos.



Evidencia 1. El script enviado en el chat original se ejecuta y muestra la alerta «Inyección de script».

3. URLs de imágenes y videos, y nueva validación en una-lib (pasos 4 y 5)

Las correcciones se hicieron en la carpeta `despues/`. El método `validarMensaje` de una-lib (alias en español de `validateMessage`) ahora recibe el mensaje JSON del cliente y devuelve un objeto seguro para mostrar, con un campo `tipo` que indica si es *texto*, *imagen* o *video*.

Reglas de validación

Entrada	Tratamiento
Texto normal o con HTML	Se escapan `& < > " ' ` /`. El script se muestra como texto y no se ejecuta.
URL de imagen	Solo `http` / `https`, analizada con `new URL()`, con extensión `jpg, jpeg, png, gif, bmp` o `webp` y sin comillas, espacios ni `< >`. Se convierte en `<img>` con la URL escapada.
URL de video	Archivos `mp4, webm, ogg` → `<video controls>`. YouTube (`youtube.com/watch`, `youtu.be`, `/embed/`) con un id válido de 11 caracteres → `<iframe>`.
URL no permitida	`javascript:`, `data:`, `ftp:`, dominios falsos de YouTube o extensiones no permitidas se tratan como texto escapado.
Nombre de usuario	Se escapa y se limita a 50 caracteres.
Color	Solo se acepta el formato hexadecimal `#RRGGBB`; si no, se usa `#000000`.
JSON inválido o vacío	Se devuelve el contenido escapado o un mensaje vacío.

Fragmento de `validarMensaje` (despues/libs/unalib.js)

```
var texto = typeof obj.mensaje === 'string' ? obj.mensaje.trim() : '';

if (this.is_valid_url_image(texto)) {
  result.tipo = 'imagen';
  result.mensaje = this.getImageTag(texto);      // <img src="URL escapada">
}
else if (this.is_valid_url_video(texto)) {
  result.tipo = 'video';
  result.mensaje = this.getVideoTag(texto);      // <video src="..." controls>
}
else if (this.is_valid_yt_video(texto)) {
  result.tipo = 'video';
  result.mensaje = this.getEmbeddedCode(texto);  // <iframe> de YouTube
}
else {
  result.mensaje = this.escapeHtml(texto);       // texto seguro
}
```

Cambios en el backend y el frontend

- **server.js:** cada mensaje del evento `Evento-Mensaje-Server` pasa por `validation.validarMensaje(msg)` antes de reenviarse. El servidor se exporta para poder probarlo, y solo escucha el puerto al ejecutar `npm start`.
- **index.html:** el nombre se inserta en su propio elemento, el color se aplica con `.css()` después de validarlo y el contenido del mensaje, ya saneado por una-lib, se muestra como texto, imagen o reproductor de video.

> Resultado: con la versión corregida, el mismo script del paso 2 aparece como texto literal en el chat y no se abre ninguna alerta. Las URLs de imágenes y videos se muestran directamente en la conversación.

4. Pruebas unitarias (pasos 6 a 8)

Primero se ejecutaron las 5 pruebas existentes del proyecto (Mocha), que pasaron. Luego las pruebas se migraron a **Jest**, la misma herramienta que se usa en las pruebas de integración. Se ampliaron a **58 pruebas** en `despues/tests/unit/unalib.test.js`, agrupadas en:

- **Validación de URLs de imágenes:** URLs válidas (jpg, png, jpeg con parámetros, webp, bmp) y rechazo de páginas HTML, dobles extensiones, `ftp:`, `javascript:`, `data:`, URLs con comillas o espacios y valores no textuales. También se verifica que la etiqueta `<img>` generada no tenga manejadores de eventos.
- **Validación de URLs de videos:** mp4, webm y ogg aceptados; avi, dobles extensiones y URLs con código rechazados. YouTube en sus formatos válidos, y rechazo de ids inválidos, dominios falsos y Vimeo. También se verifica que se genere el reproductor `<video controls>` o el `<iframe>`.
- **Prevención de inyección de scripts:** el script del paso 2 y cargas como `<img onerror>`, `<svg onload>`, `<iframe src="javascript:">`, `<a href="javascript:">` y `<body onload>`. También el nombre de usuario malicioso, el color inválido, el JSON inválido y las entradas vacías.

Comando: `npm test` (ejecuta `jest tests/unit`; la carpeta `tests/e2e` está en `testPathIgnorePatterns`).

```
PS C:\Users\Diego Araya\Downloads\lab5\despues> npm.cmd test

> una-chat@1.0.0 test
> jest tests/unit --silent

 PASS  tests/unit/unalib.test.js
  unalib
    funcion is_valid_phone
      √ deberia devolver true para 8297-8547 (4 ms)
      √ deberia devolver false para 8297p-8547 (2 ms)
    funcion is_valid_url_image
      √ deberia devolver true para http://image.com/image.jpg (2 ms)
      √ deberia devolver true para http://image.com/image.gif (1 ms)
      √ deberia devolver true para https://upload.wikimedia.org/wikipedia/commons/a/a9/Example.jpg (1 ms)
      √ deberia devolver true para https://example.com/fotos/gato.PNG (1 ms)
      √ deberia devolver true para https://example.com/a.jpeg?size=large
      √ deberia devolver true para https://example.com/a.webp (1 ms)
      √ deberia devolver true para http://example.com/a.bmp (1 ms)
      √ deberia devolver false para "https://example.com/pagina.html" (1 ms)
      √ deberia devolver false para "https://example.com/imagen.jpg.exe" (1 ms)
      √ deberia devolver false para "ftp://example.com/a.jpg"
      √ deberia devolver false para "javascript:alert(1)//a.jpg" (1 ms)
      √ deberia devolver false para ""
      √ deberia devolver false para "https://example.com/a.jpg\" onerror=\"alert(1)" (1 ms)
      √ deberia devolver false para "https://example.com/a b.jpg" (1 ms)
      √ deberia devolver false para "imagen.jpg"
      √ deberia devolver false para "" (1 ms)
      √ deberia devolver false para null (1 ms)
      √ deberia devolver false para undefined
      √ deberia devolver false para 123 (1 ms)
    funcion is_valid_url_video
      √ deberia devolver true para https://interactive-examples.mdn.mozilla.net/media/cc0-videos/flower.mp4
      √ deberia devolver true para https://example.com/videos/clip.webm (1 ms)
      √ deberia devolver true para http://example.com/clip.ogg (1 ms)
      √ deberia devolver true para https://example.com/clip.MP4?t=10 (1 ms)
      √ deberia devolver false para https://example.com/clip.avi
      √ deberia devolver false para https://example.com/clip.mp4.html
      √ deberia devolver false para javascript:alert(1)//clip.mp4 (1 ms)
      √ deberia devolver false para https://example.com/clip.mp4"><script>alert(1)</script> (1 ms)
      √ deberia devolver false para clip.mp4 (1 ms)
    funcion is_valid_yt_video
      √ deberia devolver true para https://www.youtube.com/watch?v=qYwlqx-JLok (1 ms)
      √ deberia devolver true para https://youtu.be/qYwlqx-JLok (1 ms)
      √ deberia devolver true para https://www.youtube.com/embed/qYwlqx-JLok
      √ deberia devolver true para https://m.youtube.com/watch?v=qYwlqx-JLok&t=30
      √ deberia devolver false para https://www.youtube.com/watch?v=corto (1 ms)
      √ deberia devolver false para https://www.youtube.com/watch?v=qYwlqx-JLok"><script> (1 ms)
      √ deberia devolver false para https://youtube.com.malicioso.com/watch?v=qYwlqx-JLok
      √ deberia devolver false para https://vimeo.com/123456789
    validacion de URLs de imagenes en validarMensaje
      √ convierte la URL de una imagen en una etiqueta <img> (2 ms)
      √ el <img> generado no contiene manejadores de eventos (1 ms)
    validacion de URLs de videos en validarMensaje
      √ integra un reproductor <video> con controles para un mp4 (1 ms)
      √ integra un <iframe> de YouTube con el id del video (1 ms)
```

Evidencia 2 (parte 1). Ejecución de `npm test`: pruebas de teléfono, imágenes y videos.

```
    ✓ deberia devolver true para https://example.com/clip.MP4?t=10 (1 ms)
    ✓ deberia devolver false para https://example.com/clip.avi
    ✓ deberia devolver false para https://example.com/clip.mp4.html
    ✓ deberia devolver false para javascript:alert(1)//clip.mp4 (1 ms)
    ✓ deberia devolver false para https://example.com/clip.mp4"><script>alert(1)</script> (1 ms)
    ✓ deberia devolver false para clip.mp4 (1 ms)
  funcion is_valid_yt_video
    ✓ deberia devolver true para https://www.youtube.com/watch?v=qYwlqx-JLok (1 ms)
    ✓ deberia devolver true para https://youtu.be/qYwlqx-JLok (1 ms)
    ✓ deberia devolver true para https://www.youtube.com/embed/qYwlqx-JLok
    ✓ deberia devolver true para https://m.youtube.com/watch?v=qYwlqx-JLok&t=30
    ✓ deberia devolver false para https://www.youtube.com/watch?v=corto (1 ms)
    ✓ deberia devolver false para https://www.youtube.com/watch?v=qYwlqx-JLok"><script> (1 ms)
    ✓ deberia devolver false para https://youtube.com.malicioso.com/watch?v=qYwlqx-JLok
    ✓ deberia devolver false para https://vimeo.com/123456789
  validacion de URLs de imagenes en validarMensaje
    ✓ convierte la URL de una imagen en una etiqueta <img> (2 ms)
    ✓ el <img> generado no contiene manejadores de eventos (1 ms)
  validacion de URLs de videos en validarMensaje
    ✓ integra un reproductor <video> con controles para un mp4 (1 ms)
    ✓ integra un <iframe> de YouTube con el id del video (1 ms)
  prevencion de inyeccion de scripts
    ✓ escapa el script del paso 2 del laboratorio (1 ms)
    ✓ no deja etiquetas HTML ejecutables en <img src=x onerror=alert(1)> (1 ms)
    ✓ no deja etiquetas HTML ejecutables en <svg onload=alert(1)> (1 ms)
    ✓ no deja etiquetas HTML ejecutables en <iframe src="javascript:alert(1)"></iframe> (1 ms)
    ✓ no deja etiquetas HTML ejecutables en "><script>alert(document.cookie)</script>
    ✓ no deja etiquetas HTML ejecutables en <a href="javascript:alert(1)">clic</a> (1 ms)
    ✓ el <img> generado no contiene manejadores de eventos (1 ms)
  validacion de URLs de videos en validarMensaje
    ✓ integra un reproductor <video> con controles para un mp4 (1 ms)
    ✓ integra un <iframe> de YouTube con el id del video (1 ms)
  prevencion de inyeccion de scripts
    ✓ escapa el script del paso 2 del laboratorio (1 ms)
    ✓ no deja etiquetas HTML ejecutables en <img src=x onerror=alert(1)> (1 ms)
    ✓ no deja etiquetas HTML ejecutables en <svg onload=alert(1)> (1 ms)
    ✓ no deja etiquetas HTML ejecutables en <iframe src="javascript:alert(1)"></iframe> (1 ms)
    ✓ no deja etiquetas HTML ejecutables en "><script>alert(document.cookie)</script>
    ✓ no deja etiquetas HTML ejecutables en <a href="javascript:alert(1)">clic</a> (1 ms)
    ✓ no deja etiquetas HTML ejecutables en <body onload=alert(1)> (1 ms)
    ✓ escapa el nombre del usuario (1 ms)
    ✓ rechaza un color que no sea hexadecimal (2 ms)
    ✓ conserva un color hexadecimal valido (2 ms)
    ✓ escapa un mensaje que no es JSON valido (2 ms)
    ✓ devuelve un mensaje vacio para la entrada undefined (1 ms)
    ✓ devuelve un mensaje vacio para la entrada null (1 ms)
    ✓ devuelve un mensaje vacio para la entrada "" (1 ms)
    ✓ devuelve un mensaje vacio para la entrada 42 (1 ms)
    ✓ mantiene el texto normal legible (1 ms)

Test Suites: 1 passed, 1 total
Tests:       58 passed, 58 total
Snapshots:   0 total
Time:        0.907 s, estimated 1 s
PS C:\Users\Diego Araya\Downloads\lab5\despues>
```

Evidencia 2 (parte 2). Pruebas de prevención de inyección de scripts y resumen: 58 de 58 aprobadas.

5. Pruebas de integración (paso 9)

El chat no envía los mensajes por una ruta HTTP sino por **Socket.IO**, con el evento `Evento-Mensaje-Server`. Por eso, como indica el enunciado, se instaló `socket.io-client` además de `jest` y `supertest`:

```
npm install --save-dev jest supertest socket.io-client
```

El archivo `despues/tests/integration/chat.test.js` levanta el servidor real en un puerto libre y se conecta como lo haría el navegador. Con `jest.spyOn` se comprueba que el backend llama realmente a `validarMensaje` de una-lib. Al terminar, se cierran los clientes y el servidor (`io.close()`). Casos cubiertos:

Caso	Qué se verifica
GET `/` con Supertest	Código 200, tipo `text/html` y la página del chat.
URL de imagen válida	El backend la acepta: `tipo: "imagen"` y etiqueta `<img>`; `validarMensaje` se llamó una vez.
URL de video válida (mp4)	El backend la acepta: `tipo: "video"` y reproductor `<video controls>`.
Video de YouTube	Se devuelve el `<iframe>` con el id correcto.
Script del paso 2	Se devuelve saneado como texto, sin `<script`.
URL no permitida (`javascript:`)	No se convierte en etiqueta; queda como texto sin `< >`.
Nombre de usuario con HTML	El nombre se devuelve escapado.

Comando: `npx jest tests/integration --runInBand --env=node`

```
PS C:\Users\Diego Araya\Downloads\lab5\despues> npx.cmd jest tests/integration --runInBand --env=node --silent
 PASS  tests/integration/chat.test.js
  ruta HTTP GET /
    √ responde 200 con la pagina del chat (88 ms)
  evento Socket.IO del chat con una-lib
    √ acepta una URL de imagen valida y la devuelve como <img> (18 ms)
    √ acepta una URL de video valida y la devuelve como <video> (10 ms)
    √ acepta un video de YouTube y lo devuelve como <iframe> (11 ms)
    √ sanea el script del paso 2 y lo devuelve como texto (12 ms)
    √ no convierte en etiqueta una URL no permitida (8 ms)
    √ sanea un nombre de usuario con HTML (9 ms)

Test Suites: 1 passed, 1 total
Tests:       7 passed, 7 total
Snapshots:   0 total
Time:        1.592 s, estimated 2 s
PS C:\Users\Diego Araya\Downloads\lab5\despues>
```

Evidencia 3. Pruebas de integración backend + una-lib: 7 de 7 aprobadas.

6. Pruebas end-to-end con Playwright (paso 10)

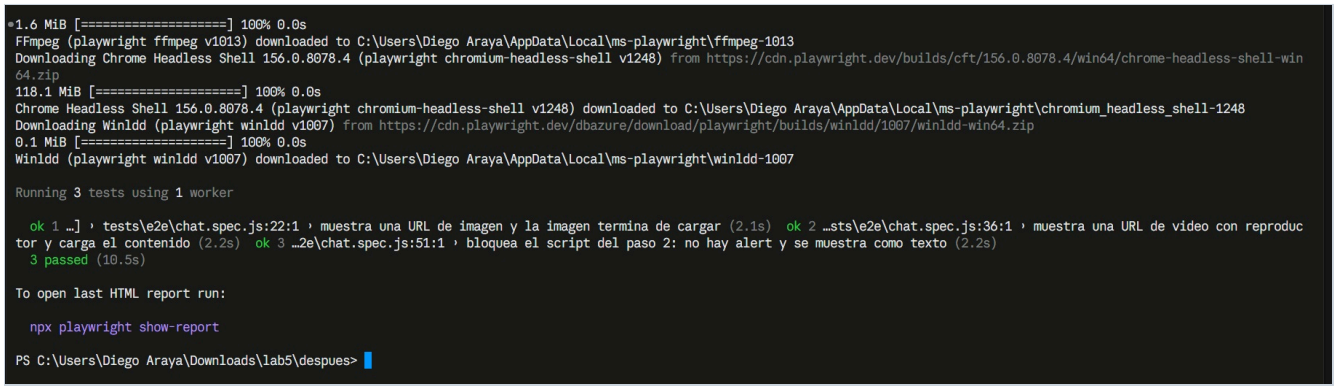
Se configuró Playwright en `despues/` con JavaScript, la carpeta `tests/e2e` y el navegador Chromium. En `playwright.config.js` se definió `testDir: './tests/e2e'` y `use.baseURL: 'http://localhost:3000'`, que es la URL real del frontend. La opción `webServer` reutiliza el servidor si ya está iniciado con `npm start`. Las pruebas corren con un solo worker porque el chat reenvía cada mensaje a todos los clientes conectados.

El archivo `despues/tests/e2e/chat.spec.js` usa `page.goto`, `getByRole`, `fill`, `click` y `expect` para tres casos:

- **Imagen:** envía una URL de Wikimedia y verifica que el `<img>` es visible y que terminó de cargar (`complete` y `naturalWidth > 0`).
- **Video:** envía un mp4 de MDN y verifica que aparece el reproductor con controles y que cargó el contenido (`readyState ≥ 1` y duración mayor que 0).
- **Script del paso 2:** registra `page.on("dialog")` antes del envío. La prueba falla si aparece cualquier alerta. Además comprueba que el mensaje se muestra como texto literal y que no se insertó ningún `<script>`.

> Como control, la prueba del script se ejecutó también contra la versión original (`antes/`) y **falló**, como se esperaba. Así se confirma que la prueba detecta la vulnerabilidad y que pasa solo con la corrección.

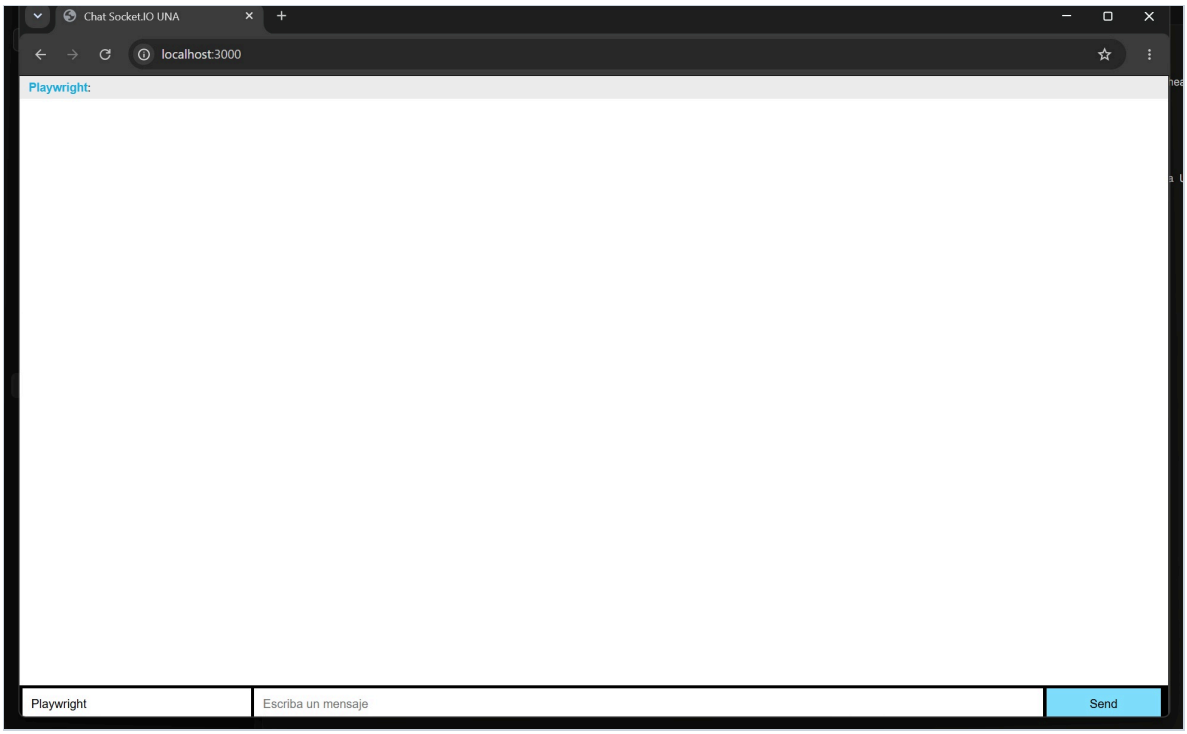
Comando: `npx playwright test --project=chromium --headed`



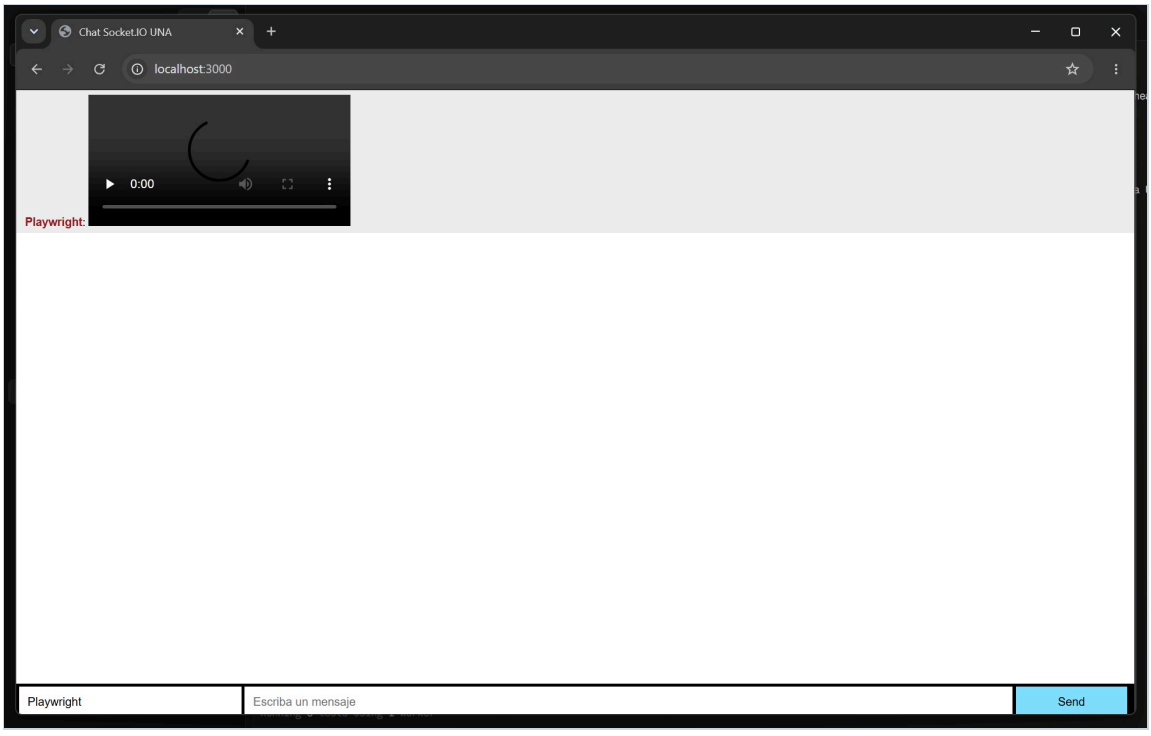
Evidencia 4. Pruebas end-to-end en Chromium: 3 de 3 aprobadas (imagen, video y bloqueo del script).

Navegador durante la ejecución (modo `--headed`)

Con `--headed`, Playwright abre una ventana visible de Chromium y escribe y envía los mensajes como lo haría un usuario. Las siguientes capturas se tomaron mientras las pruebas corrían, justo después de cada envío, por eso el contenido aparece todavía cargando. La prueba espera hasta que la imagen termina de cargar y el video obtiene sus metadatos antes de darse por aprobada.



Evidencia 4 (navegador 1). Chromium controlado por Playwright en `localhost:3000`: el usuario «Playwright» acaba de enviar la URL de la imagen.



Evidencia 4 (navegador 2). Prueba del video: el chat muestra el reproductor con controles mientras carga el mp4.

7. Resumen de resultados y conclusiones

Suite	Herramientas	Comando	Resultado
Unitarias	Jest	`npm test`	**58 / 58** aprobadas
Integración	Jest, Supertest, socket.io-client	`npm run test:integration`	**7 / 7** aprobadas
End-to-end	Playwright (Chromium)	`npm run test:e2e`	**3 / 3** aprobadas

- La aplicación original ejecutaba cualquier script enviado por un usuario. Como el chat reenvía los mensajes a todos los conectados, un atacante podía ejecutar código en el navegador de todos los demás usuarios (XSS almacenado y distribuido).
- La corrección principal es validar en el servidor: todo el texto se escapa y solo se convierten en HTML las URLs que cumplen reglas estrictas (protocolo, extensión y caracteres permitidos). El HTML de imágenes y videos lo genera la propia librería, nunca el usuario.
- Las tres suites se complementan. Las unitarias prueban la lógica de una-lib. Las de integración confirman que el backend usa esa lógica en el evento real de Socket.IO. Las end-to-end demuestran en un navegador real que el usuario ve las imágenes y videos y que el script ya no se ejecuta.

Código fuente, pruebas y configuración: https://github.com/Veldor09/lab5SI